

▼ Hollie Whitehead Anthony

Data Documentation

▼ Load Data and inspect

```
#importing libraries needed
import pandas as pd

#loading data
data = pd.read_csv('sales.csv')

#making a copy
df = data.copy()

#Intital inspection
df.head()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Prod
0	1	CA-2017-152156	08/11/2017	11/11/2017	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420.0	South	FUR-10001
1	2	CA-2017-152156	08/11/2017	11/11/2017	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420.0	South	FUR-10000
2	3	CA-2017-120000	12/06/2017	16/06/2017	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036.0	West	OFF-10000

▼ Inspecting the data for consistency and completeness

```
#Inspecting the info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9800 entries, 0 to 9799
Data columns (total 18 columns):
#   Column              Non-Null Count  Dtype  
---  -
0   Row ID              9800 non-null  int64  
1   Order ID            9800 non-null  object  
2   Order Date          9800 non-null  object  
3   Ship Date           9800 non-null  object  
4   Ship Mode           9800 non-null  object  
5   Customer ID         9800 non-null  object  
6   Customer Name       9800 non-null  object  
7   Segment             9800 non-null  object  
8   Country             9800 non-null  object  
9   City                9800 non-null  object  
10  State               9800 non-null  object  
11  Postal Code         9789 non-null  float64 
12  Region             9800 non-null  object  
13  Product ID          9800 non-null  object  
14  Category            9800 non-null  object  
15  Sub-Category        9800 non-null  object  
16  Product Name        9800 non-null  object  
17  Sales               9800 non-null  float64 
dtypes: float64(2), int64(1), object(15)
memory usage: 1.3+ MB
```

```
df.isnull().sum()
```

	0
Row ID	0
Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer ID	0
Customer Name	0
Segment	0
Country	0
City	0
State	0
Postal Code	11
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0

dtype: int64

```
# Checking the entire dataset for completeness
# This line looks for missing values (True/False) counts/sums up the missing values per column
# totals those sums and divides it by the total cells in the dataset, multiplies by 100 for %
round((df.isnull().sum().sum() / df.size) * 100,2)
```

np.float64(0.01)

▼ Postal Code appears to have wrong data type

```
#Changing Postal Code data type
df['Postal Code'] = df['Postal Code'].astype(object)
```

```
#Checking
df['Postal Code'].dtypes
```

dtype('O')

```
#Statistical Description
df['Sales'].describe()
```

	Sales
count	9800.000000
mean	230.769059
std	626.651875
min	0.444000
25%	17.248000
50%	54.490000
75%	210.605000
max	22638.480000

dtype: float64

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9800 entries, 0 to 9799
Data columns (total 18 columns):
 #   Column                Non-Null Count  Dtype  
---  --
 0   Row ID                9800 non-null  int64  
 1   Order ID              9800 non-null  object  
 2   Order Date            9800 non-null  object  
 3   Ship Date             9800 non-null  object  
 4   Ship Mode              9800 non-null  object  
 5   Customer ID           9800 non-null  object  
 6   Customer Name         9800 non-null  object  
 7   Segment               9800 non-null  object  
 8   Country               9800 non-null  object  
 9   City                  9800 non-null  object  
10   State                 9800 non-null  object  
11   Postal Code           9789 non-null  object  
12   Region                9800 non-null  object  
13   Product ID            9800 non-null  object  
14   Category              9800 non-null  object  
15   Sub-Category          9800 non-null  object  
16   Product Name          9800 non-null  object  
17   Sales                 9800 non-null  float64
dtypes: float64(1), int64(1), object(16)
memory usage: 1.3+ MB
```

```
#Checking categories for ship mode
df['Ship Mode'].value_counts()
```

	count
Ship Mode	
Standard Class	5859
Second Class	1902
First Class	1501
Same Day	538

```
dtype: int64
```

```
#Checking categories for segment
df['Segment'].value_counts()
```

	count
Segment	
Consumer	5101
Corporate	2953
Home Office	1746

```
dtype: int64
```

```
#Checking categories for Category
df['Category'].value_counts()
```

	count
Category	
Office Supplies	5909
Furniture	2078
Technology	1813

```
dtype: int64
```

```
# Checking accuracy of Customer Name Column
df['Customer Name'].str.istitle().value_counts()
```

	count
Customer Name	
True	9459
False	341

dtype: int64

```
# We seem to have inconsistencies being flagged I am checking the false values to see the problem
# Creating a variable to store the True/False return of istitle
# Showing only the False rows with ~
valid_names = df['Customer Name'].str.istitle()
df.loc[~valid_names, 'Customer Name'].head()
```

	Customer Name
70	Henry MacAllister
187	Alice McCarthy
188	Alice McCarthy
261	Dorris liebe
376	Muhammed MacIntyre

dtype: object

```
# I can see from this validation it has flagged names with 2 Uppercases in their last name, which are correct
#However I still notice that some last names are not capitalized so I am going to see if I can filter these specific ones out w

# This is saying check for first letter to be uppercase, remaining can be both, space, then check for first letter to be uppercase
valid_names_2 = df['Customer Name'].str.match(r'^[A-Z][a-zA-Z]*\s[A-Z][a-zA-Z]*$', na=False)

# Now I will recheck the invalid names with ~
df.loc[~valid_names_2, 'Customer Name']
```

	Customer Name
0	Claire Gute
1	Claire Gute
2	Darrin Van Huff
3	Sean O'Donnell
4	Sean O'Donnell
...	...
9795	Sally Hughsby
9796	Cindy Schnelling
9797	Cindy Schnelling
9798	Cindy Schnelling
9799	Cindy Schnelling

9800 rows × 1 columns

dtype: object

```
#Checking format consistencies
cols_to_check = ['Country', 'City', 'State', 'Region']

for col in cols_to_check:
    print(f'\n{col}')
    print(df[col].str.istitle().value_counts())
```

```
Country
Country
True    9800
Name: count, dtype: int64
```

```
City
```

```

City
True      9800
Name: count, dtype: int64

State
State
True      9790
False      10
Name: count, dtype: int64

Region
Region
True      9800
Name: count, dtype: int64

```

```

#Further investigating the state column from the indication of 10 False outputs
valid_state = df['State'].str.istitle()
df.loc[~valid_state, 'State'].head(10)

```

```

      State
1020  District of Columbia
1021  District of Columbia
1022  District of Columbia
1638  District of Columbia
1639  District of Columbia
1640  District of Columbia
1641  District of Columbia
1845  District of Columbia
1846  District of Columbia
4596  District of Columbia

```

dtype: object

```

#Checking OrderId format, it appears the format is derived from the Country abbreviation but we are seeing CA with no Canada re
df['Order ID'].str[:2].value_counts()

```

```

      count
Order ID
CA      8161
US      1639

```

dtype: int64

Data Quality Metrics Summary

1. Consistency:

With the consistency validation checks on Customer Name, Country, State, Region and City. I have noticed that even though there are no values missing, there are a lot of different name variations making it difficult to standardize the format consistently with Title Case style. My recommendation would be to standardize text fields to lowercase across the dataset as a lot of fields are related, so this formatting issue follows into other fields as well. This inconsistency could create fragmented or inconsistent analysis results.

With consistency validation checks from OrderID I see we have 2 formats of CA and US in the beginning. This resembles a reference to countries Canada and United States, however we only have United States listed from our Country field. I would need formatting verification sources to fully validate this inconsistency. I would

recommend verification before doing any analysis to confirm the Orders are in fact from United States and not Canada

✓ **2.Completeness:**

Completeness: This dataset is 99.99% complete.

✓ **3. Accuracy:**

I could not verify accuracy with no original source of the data, however I would recommend verifying the Order ID's unique identifier as this inconsistency could be related to an inaccurate data extraction.

✓ **4. Timeliness**

Could not verify this data was up to date